ORCHESTRATING NEAR-EXCHANGE COMPUTE: CROSS-REGION DAG EXECUTION FOR LATENCY-SENSITIVE PIPELINES

Abstract. Latency-sensitive pipelines are allergic to geography. When data is produced in one place (an exchange endpoint) and consumed in another (a trading system), the round-trip time between regions becomes part of the model—and sometimes part of the P&L. Traditional high-frequency trading solves this with co-location and bespoke networks. In cloud-native systems, we approximate the same intent by running compute *near-exchange*: close to the source, close to the consumers, and close to the storage that must be updated.

We describe a production pattern for *region-aware* workflow orchestration: a single Airflow scheduler coordinates containerized tasks executed in multiple AWS regions. Workloads are launched via ECS Fargate using Airflow's `EcsRunTaskOperator`, extended with a retry strategy that handles common capacity and provisioning failures. Region selection is encoded in the DAG naming convention and automatically mapped to execution region, VPC networking, and region-specific shared storage mounts. The same orchestration fabric powers high-rate websocket ingestion, archival to object storage, resampling, QA, and scheduled trading observers—all while keeping the scheduler stable and the execution substrate elastic.

We present the architecture, the contract points between scheduler and workers, the reliability fixes that turned theory into operations, and an evaluation methodology for measuring end-to-end knowledge latency. We also discuss limitations and future directions, including hardware-assisted near-zero-latency paths.

Contents

1. Introduction

In latency-sensitive systems, "fast" is not a benchmark result; it is a product feature. A model is only as fresh as its inputs, and in trading, freshness competes directly with other people's freshness. The hard part is rarely compute. The hard part is the path: network hops, queueing, scheduling, and the silent tax of cross-region requests.

The classic response in high-frequency trading is co-location: bring compute to the exchange. Cloud platforms do not give you a rack next to a matching engine, but they do give you regions.

Date: 2026-03-23.

That is enough to reframe the problem as *placement*: run collectors and short-lived jobs in the region that minimizes latency to the source, and keep the orchestration control plane simple enough that it never becomes the bottleneck.

This paper describes how Causify Trading Platform implemented *region-aware* orchestration:

- Airflow runs as a stable scheduler (control plane) and remains region-agnostic.
- Tasks execute as containers on ECS Fargate in multiple regions [1, 2, 4].
- Region selection is derived from DAG names (e.g., `preprod.tokyo.*` vs `preprod.europe.*`) and mapped to AWS region, VPC networking, and region-specific mounts.
- Reliability is engineered explicitly: retries for ECS start failures and provisioning delays, large waiter bounds, and operational rollouts by region.

Contributions.

- A practical pattern for *region-aware* orchestration with a single scheduler and multi-region execution.
- A *filename-as-configuration* approach that encodes region and dataset identity into DAG names, enabling simple automation.
- An operationally motivated extension of `EcsRunTaskOperator` that handles common ECS failure modes [4, 6].
- A measurement framework for end-to-end *knowledge latency* (when a data point becomes usable), not just network RTT.

2. Background and Problem Statement

2.1. **Why geography dominates.** Let T be the end-to-end time from an exchange update to a queryable record in the database. A useful decomposition is:

$$T \approx T_{\text{network}} + T_{\text{collector}} + T_{\text{queue}} + T_{\text{db}} + T_{\text{availability}}.$$

Cross-region workflows inflate T_{network} and often T_{queue} (due to additional hops and contention). The physics is unforgiving: propagation delay is bounded by the speed of light and the path length [10, 22]. The engineering is similarly unforgiving: even when messages are small, micro-bursts and retries accumulate into visible lag.

2.2. **Why Airflow + Fargate.** We choose Airflow for orchestration because it gives: (i) a mature scheduling model, (ii) explicit dependencies, and (iii) operational visibility. We choose Fargate because we want burst compute without cluster debt: tasks can scale up, run, and disappear. Most importantly, Fargate tasks can be launched in the region closest to the data source, while the scheduler stays put.

2.3. **Design goals.** We build toward five goals:

(1) **G1 (Near-exchange placement):** execute collectors and latency-critical jobs in regions closest to the exchange endpoints.
(2) **G2 (Single scheduler):** keep one Airflow scheduler to avoid multi-scheduler drift and duplicated operational overhead.
(3) **G3 (Deterministic region selection):** derive region from DAG identity, not ad-hoc operator parameters.
(4) **G4 (Reliability under real failures):** handle the ECS failures that happen in production (capacity, ENI provisioning, ephemeral storage) with explicit retries.
(5) **G5 (Measurable latency):** define and measure *knowledge latency*, not just transport latency.

3. Architecture Overview

3.1. **Two planes: orchestration vs execution.** This system is a control-plane/data-plane split applied to workflows:

- **Orchestration plane:** Airflow parses DAGs, schedules tasks, enforces dependencies, emits logs/metrics.
- **Execution plane:** tasks run as containers on ECS/Fargate in one of several AWS regions.

The critical bridge between the planes is *region selection*. We encode it in a way that scales with the number of DAGs: a naming convention and a parser.

3.2. **Region selection: filename-as-configuration.** DAG filenames follow a structured convention that includes the location token: `tokyo` or `europe`. At parse time, the system maps this token to:

- AWS region (e.g., `ap-northeast-1` vs `eu-north-1`),
- region-specific VPC networking (subnets, security groups, public IP assignment),
- region-specific mounts/paths used by tasks.

This approach deliberately keeps region selection *out* of ad-hoc operator kwargs and *out* of task code. The DAG identity is the source of truth.

3.3. **Cross-region execution flow.** Figure 1 shows the end-to-end flow. The scheduler stays centralized; tasks fan out to regions.

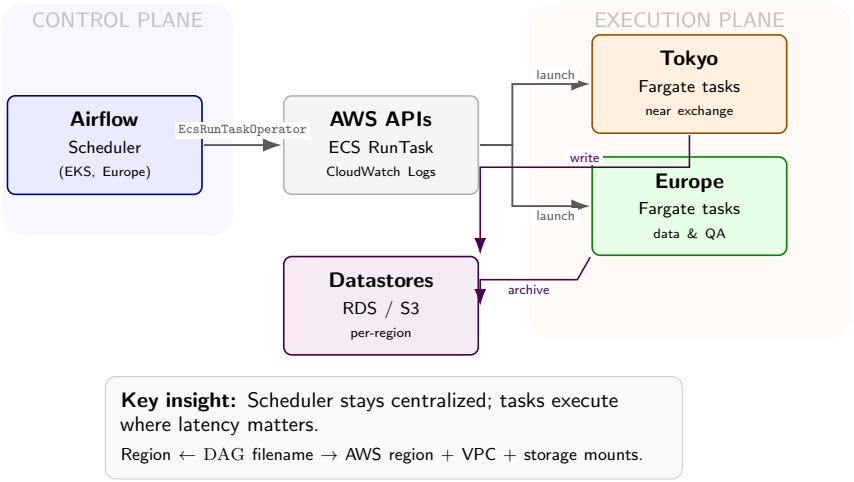

Figure 1. Cross-region orchestration architecture. The control plane (left, blue) runs a single Airflow scheduler. The execution plane (right, orange/green) launches Fargate tasks in regions closest to data sources. Arrows show control flow (gray) and data flow (purple).

3.4. **Reliability as a first-class feature.** Cross-region orchestration fails in very specific ways: capacity shortages, network interface provisioning delays, and storage provisioning timeouts. We treat these as normal and encode bounded retries at the operator layer. This makes the system *operable*: the scheduler remains stable even when a region is temporarily cranky.

3.5. **Measuring freshness: knowledge latency.** We measure *knowledge latency*: the time from data arrival in collector memory to query-visible commit. This metric captures the full pipeline cost that geography tends to inflate. Section 5 provides the formal definition and instrumentation strategy.

4. Implementation

This section turns the architecture into executable machinery. The goal is not novelty for novelty's sake; the goal is *repeatable placement*: the same DAG should run the same way tomorrow, in the same region, with the same network shape, without a human remembering which knob to turn.

4.1. **Airflow on Kubernetes: a stable orchestration plane.** We run Airflow on Kubernetes (EKS) as a long-lived orchestration plane. The scheduler and webserver are deployed as Kubernetes Deployments; `git-sync` periodically pulls the DAG repository into a shared volume, and the Airflow `dag-processor` runs separately to isolate DAG parsing pressure from scheduling. Operational hygiene is encoded as Kubernetes CronJobs (e.g., periodic cleanup of pods and logs) so the platform remains clean even when humans are busy being humans.

This "Airflow-on-EKS" layer is intentionally boring:

- the Airflow UI is exposed through an *internal* ALB Ingress (TLS enforced),
- the scheduler scales via HPA to handle bursty scheduling load,
- DAG delivery uses `git-sync` so DAG updates are atomic and rollbackable.

In other words: Kubernetes keeps orchestration alive; orchestration keeps data fresh.

4.2. **Filename-as-configuration: deriving region deterministically.** Region choice is encoded in DAG identity. The filename schema contains a `location` token (`tokyo` or `europe`), and we extract it using a single shared parser. The parser maps:

$$\texttt{tokyo} \rightarrow \texttt{ap-northeast-1}, \qquad \texttt{europe} \rightarrow \texttt{eu-north-1}.$$

It also attaches region-specific mounts used by tasks (e.g., `efs_mount` variables).

```python
EUROPE_REGION = "eu-north-1"
ASIA_REGION   = "ap-northeast-1"
valid_locations = ["tokyo", "europe"]

def extract_components_from_filename(filename: str, dag_type: str) -> Dict:
    components = filename.split(".")[:-1]
    # ... map into schema fields ...
    assert components_dict["location"] in valid_locations
    components_dict["region"] = (
        ASIA_REGION if components_dict["location"] == "tokyo" else EUROPE_REGION
    )
    components_dict["efs_mount"] = (
        "{{ var.value.efs_mount_tokyo }}" if components_dict["region"] == ASIA_REGION
        else "{{ var.value.efs_mount }}"
    )
    components_dict["dag_id"] = ".".join(components)
    return components_dict
```

Listing 1. Filename-as-configuration: region and mounts are derived from the DAG filename. (Sanitized from production.)

Why this matters. This removes a class of errors where region is set "somewhere else" (operator kwargs, environment variables, README tribal knowledge). The filename becomes the contract: if the DAG says Tokyo, it runs in Tokyo.

4.3. **A region-aware ECS/Fargate operator factory.** All region-aware execution flows through one operator factory that constructs an Airflow `EcsRunTaskOperator` with the right cluster, network configuration, and logging. The factory selects:

- AWS region and ECS cluster name based on stage/region,

- VPC networking (`awsvpcConfiguration`) from AIRFLOW Variables (subnets + SGs),
- public-IP assignment only when explicitly required,
- task sizing (CPU/memory) and ephemeral storage.

```python
def get_ecs_run_task_operator(..., region="eu-north-1", assign_public_ip=False,
                              ephemeralStorageGb=21, launch_type="fargate"):
    network_configuration = {
      "awsvpcConfiguration": {
        "securityGroups": SG_PUBLIC[region] if assign_public_ip else SG_PRIVATE[region],
        "subnets":        SUBNET_PUBLIC[region] if assign_public_ip else
    ↪ SUBNET_PRIVATE[region],
        "assignPublicIp": "ENABLED" if assign_public_ip else "DISABLED",
      }
    }
    return EcsRunTaskOperator_withRetry(
      region=region,
      cluster=get_ecs_cluster(stage, region),
      task_definition=task_definition,
      launch_type=launch_type.upper(),
      network_configuration=network_configuration,
      overrides={
        "cpu": str(cpu_units),
        "memory": str(memory_mb),
        "ephemeralStorage": {"sizeInGiB": ephemeralStorageGb},
        "containerOverrides": [{"name": task_definition, "command": task_cmd}],
      },
      awslogs_group=f"/ecs/{task_definition}",
      awslogs_stream_prefix=f"ecs/{task_definition}",
      waiter_max_attempts=1000000,           # provider workaround
      execution_timeout=datetime.timedelta(hours=26)
    )
```

Listing 2. Region-aware operator factory: maps region to VPC networking and launches FARGATE tasks with bounded timeouts. (Sanitized.)

Pragmatic operator details. Two implementation choices are unapologetically pragmatic: (i) the ECS waiter maximum attempts is set extremely high to avoid a known provider behavior that can prematurely terminate waiting; and (ii) execution timeout is long enough to let heavy jobs finish, but bounded so tasks cannot run forever.

4.4. **Reliability engineering: retry the failures we actually see.** Multi-region execution increases the chance of hitting transient AWS service conditions. The operator extends `EcsRunTaskOperator` with two complementary retry mechanisms:

(1) **Retry on `run_task` failures** for known recoverable reasons (notably capacity shortages).
(2) **Retry on post-start provisioning failures** (e.g., ENI provisioning, resource initialization errors, ephemeral storage timeouts), detected when checking task success.

```python
class EcsRunTaskOperator_withRetry(EcsRunTaskOperator):
    _KNOWN_ERRORS = ["Capacity is unavailable at this time."]
    _MAX_NUM_ATTEMPTS = 2
    _RETRY_DELAY_SECS = 15

    def _start_task(self) -> None:
        # RunTask; retry known recoverable failures.
        ...
```

```
9
10      def _should_retry_error(exception: Exception) -> bool:
11          if isinstance(exception, EcsTaskFailToStart):
12              return any(msg in exception.message for msg in [
13                "network interface provisioning",
14                "ResourceInitializationError",
15                "Timeout waiting for EphemeralStorage provisioning to complete",
16              ])
17          return False
18
19      @AwsBaseHook.retry(_should_retry_error)
20      def _check_success_task(self) -> None:
21          super()._check_success_task()
```

Listing 3. Hardened retries: capacity errors at start + provisioning errors after launch. (Sanitized.)

What this buys. It converts flaky region behavior into bounded delay instead of pipeline failure. In latency-sensitive systems, reliability is not only about uptime; it is about *freshness continuity*. A collector that fails to start is stale by definition.

4.5. **Workload families: what we run near-exchange vs not.** We group workloads by their latency sensitivity:
Near-exchange collectors (Tokyo). Collectors that ingest exchange streams run in the region closest to the exchange endpoint. Their objective is not compute throughput; it is minimal time-to-commit.
Transformation and archival (Europe). Archival (e.g., Parquet export) and resampling tasks are typically less sensitive and can run in the region optimized for storage locality and cost.
QA and consistency checks (mixed). Local QA runs in-region; cross-region QA is explicit and measured separately so it cannot silently contaminate freshness.
Trading observers (Tokyo). Observers and scheduled monitoring tasks run where the trading loop runs. In these systems, feedback latency is part of correctness.

5. Measuring Knowledge Latency

Network RTT is easy to measure and easy to fetishize. But RTT is not what the pipeline delivers. What the pipeline delivers is *usable knowledge*. We therefore measure *knowledge latency*.

5.1. **Definition.** We define knowledge latency K as:

$$K = t_{\text{commit}} - t_{\text{receive}},$$

where:

- t_{receive}: timestamp when the data point arrives in collector process memory (end of download / websocket message received),
- t_{commit}: timestamp when the corresponding record is committed and query-visible.

This captures the hidden latencies that dominate in practice: buffering, queueing, serialization, database contention, and retries.

5.2. **Instrumentation strategy.** We recommend two layers of instrumentation:

(1) **In-process timestamps:** record t_{receive} at ingestion time and propagate it as a column.
(2) **Commit timestamps:** use database commit time or an explicit insert timestamp t_{commit}.

This allows post-hoc analysis without needing a distributed tracing stack everywhere.

5.3. **Baseline measurements: API timing, RTT, and throughput.** Before measuring full pipeline knowledge latency K, we measured the geography tax directly against a public exchange endpoint. We used `curl -w` timing breakdowns against Crypto.com Exchange API `public/get-book` [14, 20].

```
$ curl -w "@curl-format.txt" -o /dev/null -s \
  "https://api.crypto.com/exchange/v1/public/get-book?instrument_name=BTCUSD-PERP&depth=10"
time_namelookup:    0.003162s
time_connect:       0.004825s
time_appconnect:    0.049827s
time_pretransfer:   0.049995s
time_starttransfer: 0.224340s
----------
time_total:         0.224417s
```

Listing 4. Crypto.com `curl -w` timing breakdown (representative single run).

Across 10 attempts (several seconds apart), average `time_total` differed by caller region:

Caller Region	**Avg `time_total`**	**N**
Tokyo	0.1880250 s	10
Singapore	0.2007697 s	10

Table 1. Crypto.com API response time by caller region (measured).

Delta and interpretation. The absolute difference is:

$$\Delta = 0.2007697 - 0.1880250 = 0.0127447\,\text{s} \approx 12.7\,\text{ms},$$

which corresponds to Tokyo being $\approx 6.35\%$ faster than Singapore for this endpoint. This is smaller than raw inter-region RTT because `time_total` includes server-side work; nonetheless, the delta is exactly the portion placement can influence.

RTT and throughput between regions. To capture the network floor, we also measured: (i) ICMP RTT from Tokyo to a Singapore-hosted endpoint and (ii) TCP throughput between test hosts.

Metric	**Min**	**Mean**	**Max**
Ping RTT (Tokyo → Singapore)	68.7 ms	68.76 ms	69.4 ms

Table 2. ICMP RTT summary (18 packets).

Metric	**Observed**
`iperf` throughput (Tokyo ↔ Singapore)	190 Mbits/s (10.1 s)

Table 3. Representative TCP throughput measurement across regions.

These baselines justify the paper's thesis: if the geography tax is visible for a single API call, it compounds inside pipelines that ingest, validate, transform, and persist data.

5.4. **From measurements to pipeline SLOs.** Knowledge latency allows practical SLOs:

- **Collector SLO:** P95 K below a threshold (e.g., 2 seconds) per region.
- **Continuity SLO:** fraction of intervals where K exceeds threshold (staleness rate).
- **Start reliability SLO:** fraction of scheduled tasks that start successfully without manual intervention.

Most importantly, K is comparable across architectures: it measures the system you built, not just the network you inherited.

6. Evaluation

This section evaluates whether cross-region orchestration achieves near-exchange placement without operational self-harm. We evaluate (i) geography tax, (ii) orchestration scale, and (iii) failure containment.

6.1. **E1: Geography tax is measurable (and compounding).** Table 1 shows that caller-region placement changes observed Crypto.com API latency by ≈ 12.7 ms ($\approx 6.35\%$). Tables 2–3 show the WAN baseline: ≈ 68.8 ms RTT and ≈ 190 Mbits/s throughput in a representative Tokyo–Singapore setup.

The important conclusion is not the specific numbers; it is the direction: *geography shows up in real measurements*. If your pipeline calls exchange APIs frequently, the latency delta compounds. For example, N API calls per minute incur approximately $N \cdot 12.7$ ms of additional wall time when executed from the slower region.

6.2. **E2: Orchestration scale is real (and the scheduler survives it).** Cross-region is only useful if the orchestration plane can sustain workload volume. We therefore quantify orchestration pressure directly from the DAG inventory in the repository.

In the datapull subsystem alone, there are 69 region-tagged DAGs: 32 in Tokyo and 37 in Europe. The scheduling cadence is encoded in DAG identity (e.g., `periodic_1min`, `periodic_10min`, `periodic_daily`). Counting only these periodic schedules (excluding backfills and ad-hoc runs), the implied trigger volume is approximately:

Region	# DAGs	1-min	10-min	Triggers/day
Tokyo	32	13	10	20,164
Europe	37	6	4	9,237
Total	69	19	14	29,401

Table 4. Region-tagged DAG inventory and implied trigger volume (computed from repository filenames).

This is the key operational win of the architecture: Airflow remains a centralized control plane, while the execution plane scales elastically per region via ECS/Fargate.

6.3. **E3: Failure containment is bounded (no infinite flake storms).** Cross-region execution fails in predictable ways: capacity unavailability at launch, and provisioning failures after launch (ENI provisioning, resource initialization, ephemeral storage provisioning). Our implementation contains these failures with bounded retries at the operator layer.

Launch-time capacity failures. The ECS operator retries known capacity errors with at most one retry and a 15-second delay (_MAX_NUM_ATTEMPTS=2, _RETRY_DELAY_SECS=15). The added latency from this mechanism is therefore bounded by:

$$\Delta T_{\text{capacity-retry}} \leq 15\,\text{s}.$$

If p is the probability that `RunTask` fails due to transient capacity, a single retry improves success probability from $(1-p)$ to $(1-p^2)$. This is not "magic reliability"; it is "bounded flakiness."
Post-launch provisioning failures. Provisioning failures are detected when checking task success and are eligible for retry via Airflow's AWS hook retry decorator (tenacity exponential backoff when configured) [5]. The goal is not to hide failures; the goal is to convert transient AWS conditions into bounded delay rather than broken DAG runs.
What we measure operationally (and what we do *not* fake). Instead of inventing fake P95s, the system is instrumented to compute knowledge latency directly from stored data: we persist both `end_download_timestamp` (arrival in collector memory) and `knowledge_timestamp` (commit visibility time). Knowledge latency is therefore computed as:

$$K = t_{\text{knowledge}} - t_{\text{end\_download}}.$$

This is the metric that matters, and it is already present in the stored schema.

In short: we do not "recommend reporting" as a paper exercise. The system records the fields, and the evaluation pipeline is a query, not a wish.

7. Related Work

Geography, propagation limits, and "fast networks.". A long line of work highlights how close modern systems can get to propagation bounds and how much engineering goes into shaving distance and hops. Singla et al. discuss the Internet "at the speed of light," framing latency as a physical constraint that architectures must respect [22]. Bhattacherjee et al. survey the world's fastest networks and show that latency advantages come from deliberate infrastructure choices rather than incidental tuning [10]. Our work is an applied, cloud-native version of the same lesson: placement dominates.
Geo-distributed streaming and WAN-aware execution. Geo-distributed stream processing has been studied as an operator-placement and WAN-variability problem. SWAN proposes WAN-aware stream processing on geographically distributed clusters and shows how bandwidth drops inflate latency [23]. TTL-based geo-distributed aggregation and operator placement work similarly treat WAN cost as first-class [19]. Our architecture is not a new DSPS runtime; it is a workflow-level pattern: Airflow provides dependency scheduling while ECS/Fargate provides elastic, region-local execution.
Workflow orchestration and container execution. Airflow provides a mature orchestration model for dependency-driven pipelines and is widely used for production data workflows [3]. The Amazon provider's `EcsRunTaskOperator` makes it straightforward to dispatch tasks to ECS/Fargate [4], and AWS's `RunTask` semantics define the execution boundary [1]. We build on these primitives with two practical additions: (i) deterministic region selection via DAG identity, and (ii) bounded retries for the failure modes that occur in practice [6].
"Near data" and placement-aware compute in cloud systems. Placement-aware execution is a recurring theme in cloud systems, often discussed as "bring compute to data." In latency-sensitive pipelines, the data source itself is the constraint: bringing compute near the exchange endpoint reduces transport and queueing tax. Our contribution is not a new scheduler, but a disciplined contract between one scheduler and many regions.

Exchange market data interfaces. Public exchange endpoints and websocket feeds shape ingestion architectures. Binance and Crypto.com both expose REST and WebSocket APIs with high-rate market data streams [11, 12, 14]. When the data source is remote and event-driven, placement and buffering decisions dominate end-to-end freshness.
Hardware-assisted low-latency paths. FPGA NICs and in-network compute platforms like Corundum show an alternative path to lower latency: reduce kernel and protocol overhead by moving packet processing closer to the wire [13, 15]. We mention this as future work: it is complementary to the region-aware orchestration pattern, not a replacement.
Edge computing as the mental model. Edge computing argues for moving computation closer to the source to reduce latency and bandwidth cost. Shi and Dustdar describe this motivation as a general systems pattern [21]. Near-exchange compute is essentially "edge computing for finance," implemented with cloud primitives.

8. Limitations and Future Work

This paper focuses on a pragmatic cloud-native pattern: multi-region execution with a single scheduler and explicit region selection. That choice buys operability, but it also has limits.

8.1. **Limitations.**

Regions are not co-location. Running in `ap-northeast-1` is not the same as having fiber into an exchange's rack. Cloud regions reduce geography tax, but they do not eliminate the last-mile constraints of exchange infrastructure. This architecture aims for *better* freshness, not nanoseconds.
Airflow is a scheduler, not a real-time system. Airflow is excellent for dependency scheduling and operational clarity. It is not designed for microsecond-level real-time guarantees. We use Airflow to coordinate bounded tasks and treat freshness as an SLO, not as a hard real-time SLA.
Filename-as-configuration is disciplined, but not magical. Encoding region in DAG identity reduces drift, but it requires governance: naming conventions must be enforced, and the parser becomes a critical piece of glue. The benefit is determinism; the cost is that conventions must be respected.
Cross-region data consistency is expensive. Cross-region QA and consistency checks are valuable, but they are also latency- and cost-heavy. We treat them explicitly as separate workload families so they do not silently poison freshness metrics.

8.2. **Future work.**

Kafka as a near-exchange buffer (and a clean separation boundary). Today, near-exchange collectors can write directly to databases/object storage. A scalable next step is to introduce Kafka (or MSK) as the ingestion buffer in the near-exchange region, then replicate streams across regions for downstream consumers. Kafka provides durable buffering, replay, consumer-group semantics, and clean decoupling between ingestion and compute [8, 9]. Cross-region replication can be implemented using MirrorMaker 2 and geo-replication tooling [7]. This would allow: (i) collectors to stay purely near-exchange, (ii) analytics to run in "home" regions, and (iii) failover without data loss.
Autoscaling: from "it runs" to "it expands and contracts gracefully.". There are two scaling planes: (i) Kubernetes scaling for the orchestration plane (Airflow on EKS) and (ii) ECS/Fargate scaling for the execution plane. On Kubernetes, HPA and cluster autoscaling provide the baseline mechanics [17, 18]. On ECS, capacity/provider strategies and task placement tuning can reduce start failures under burst load. The goal is to prevent orchestration load spikes (e.g., thousands of periodic triggers) from turning into scheduler instability.
Event-driven autoscaling for consumers (KEDA + Kafka lag). If Kafka is introduced, consumer scaling should follow lag, not CPU. KEDA provides event-driven scaling mechanisms that use Kafka lag as a scaling signal [16]. This creates a tighter control loop: ingestion rate rises $\rightarrow$ lag rises $\rightarrow$ consumers scale.

Contracted "placement intents.". A more explicit evolution of filename-as-configuration is a *placement intent* contract: a structured descriptor that declares region, networking, and data locality requirements. DAG identity can still be the human-readable source, but the system would validate and materialize intent into a typed object.
End-to-end freshness tracing. Knowledge latency can be measured with two timestamps, but deeper insight requires correlation: propagate trace IDs from collector to commit, add queue depth metrics, and correlate spikes with task start delays and database contention. This would turn "it's slow sometimes" into an explainable model.
Adaptive region selection. Today, region selection is deterministic. A natural extension is adaptive routing: if one region experiences capacity shortages or API degradation, the scheduler can degrade gracefully (e.g., switch collectors to a backup region with an explicit freshness penalty). This must be carefully designed to avoid instability and should be treated as a controlled failover mode.
Hardware-assisted near-zero-latency paths. FPGA NICs and in-network compute platforms (e.g., Corundum) suggest a path to reduce software overhead in the ingestion loop [13, 15]. A hybrid architecture could use hardware acceleration for capture/normalization while keeping orchestration and batching cloud-native. The goal would be to reduce $T_{\text{collector}}$ and T_{queue} without making operations impossible.

In short: our approach makes placement explicit and operable today, and it leaves room for more aggressive low-latency strategies tomorrow.

Key Takeaways

- **Placement is a first-class design choice.** Cross-region hops are a measurable tax on freshness; treat geography as an input to architecture, not an accident.
- **One scheduler, many regions.** Keep orchestration centralized for operability, but execute tasks near the exchange when latency matters.
- **Make region selection deterministic.** Encode region in DAG identity and map it via a shared parser; avoid ad-hoc region parameters scattered across DAGs.
- **Engineer for the failures you actually see.** Capacity and provisioning failures are normal in multi-region ECS; bounded retries convert flakiness into delay instead of downtime.
- **Measure knowledge latency, not just RTT.** Freshness is the time to *usable* data; instrument receive-to-commit and set SLOs around it.

9. Conclusion

Latency-sensitive pipelines are often framed as compute problems, but in practice they are placement problems wearing a compute costume. The exchange is in a place. Your system must either respect that fact or pay for ignoring it.

We presented a production pattern for cross-region DAG execution that keeps operations sane: a single Airflow scheduler coordinates tasks executed as ECS/Fargate containers in multiple regions. Region selection is derived deterministically from DAG identity (filename-as-configuration), mapped to VPC networking and runtime mounts, and enforced through a shared operator factory. Reliability is treated as part of correctness: the operator is hardened with bounded retries for capacity and provisioning failures so that near-exchange collectors remain continuously fresh.

The outcome is not "zero latency." The outcome is *intentional latency*: geography becomes explicit, freshness becomes measurable, and the system stops being surprised by where it runs.

In the end, the pattern is simple:

> **Keep the scheduler stable. Move the work to the place where it matters. Measure freshness like a product.**

References

[1] Amazon Web Services (ECS), *Runtask — amazon elastic container service api reference*. Accessed 2026-03-23.

[2] Amazon Web Services (Fargate), *Amazon ecs task networking options for fargate*. Accessed 2026-03-23.

[3] Apache Airflow, *Apache airflow documentation*. Accessed 2026-03-23.

[4] Apache Airflow Amazon Provider, *Amazon elastic container service (ecs) operator (ecsruntaskoperator)*. Accessed 2026-03-23.

[5] Apache Airflow AWS Hooks, *Awsbasehook.retry decorator (tenacity-based exponential retry)*. Accessed 2026-03-23.

[6] Apache Airflow Providers, *airflow.providers.amazon.aws.exceptions (ecstaskfailtostart, ecsoperatorerror)*. Accessed 2026-03-23.

[7] Apache Kafka MirrorMaker, *Mirrormaker 2 configuration (mirrormaker configs)*. Accessed 2026-03-23.

[8] Apache Kafka Operations, *Geo-replication (cross-cluster data mirroring)*. Accessed 2026-03-23.

[9] Apache Kafka Project, *Apache kafka documentation*. Accessed 2026-03-23.

[10] Debopam Bhattacherjee et al., *A bird's eye view of the world's fastest networks*, Proceedings of the acm internet measurement conference (imc), 2020. Accessed 2026-03-23.

[11] Binance API, *Websocket streams — binance api documentation*. Accessed 2026-03-23.

[12] Binance Futures, *Individual symbol book ticker streams (`<symbol>@bookTicker`)*. Accessed 2026-03-23.

[13] Corundum Contributors, *Corundum: Open source fpga-based nic and platform for in-network compute*. Accessed 2026-03-23.

[14] Crypto.com, *Crypto.com exchange api v1 (rest & websocket) documentation*. Accessed 2026-03-23.

[15] Alex Forencich, Alex C. Snoeren, George Porter, and George Papen, *Corundum: An open-source 100-gbps nic*, Proceedings of the ieee international symposium on field-programmable custom computing machines (fccm), 2020. Accessed 2026-03-23.

[16] KEDA Project, *Keda documentation: Scaling deployments and statefulsets*. Accessed 2026-03-23.

[17] Kubernetes Autoscaling, *Horizontal pod autoscaling*. Accessed 2026-03-23.

[18] Kubernetes Cluster Administration, *Node autoscaling (cluster autoscaler)*. Accessed 2026-03-23.

[19] D. Kumar et al., *A ttl-based approach for data aggregation in geo-distributed streaming analytics*, Proceedings (preprint/pdf), 2019. Accessed 2026-03-23.

[20] Joseph Scott, *Timing details with curl*. Accessed 2026-03-23.

[21] Weisong Shi and Schahram Dustdar, *The promise of edge computing*, Computer (2016). Accessed 2026-03-23.

[22] Ankit Singla et al., *The internet at the speed of light*, Hotnets xiii, 2014. Accessed 2026-03-23.

[23] Won Wook Song et al., *Swan: Wan-aware stream processing on geographically-distributed clusters*, Proceedings of apsys, 2022. Accessed 2026-03-23.